

SONOFF ZBDongle-E V2

Router / Repeater Flashing Report

Date 25 September 2026

Prepared by Syed

Project Awesom Living — Zigbee Mesh Extension

Organization VAYUZ Technologies, Noida

1. Objective

Flash a **spare** SONOFF Zigbee 3.0 USB Dongle Plus V2 (ZBDongle-E) with **Zigbee router firmware** so it can serve as a dedicated mesh relay/repeater. This will extend Zigbee range for battery-powered sensors (sleepy end devices) that currently sit at the edge of the coordinator's radio range.

Once flashed, the dongle will be powered by a standard USB charger and placed midway between the coordinator and distant sensors — no host computer needed, just power.

2. Dongle Hardware Details

Verified from physical unit — label information and internal specifications.

2.1 Label Information (from dongle back)

Model	ZBDongle-E
Input	5V, 100mA Max
Wireless	Zigbee 3.0
FCC ID	2APN6SZBD-E
IC ID	27875-ZBDE
Serial Number	SL406010FC
Manufacturer	Shenzhen Sonoff Technologies Co., Ltd
Address	BF.6F, BldgA, No.663, Bulong Rd, Shenzhen, Guangdong, China

Certifications	FCC, CE, WEEE
----------------	---------------

2.2 Internal Specifications

Product Name	SONOFF Zigbee 3.0 USB Dongle Plus V2 (ZBDongle-E)
Wireless Chip	Silicon Labs EFR32MG21 (Zigbee 3.0 / Thread)
USB-Serial Chip	WCH CH9102 (NOT CH340/CH341)
Antenna	External SMA omnidirectional (attached)
Housing	Black aluminium heatsink-finned body
USB Connector	USB-A male (direct plug, no cable)
Current Firmware	Factory default (EmberZNet coordinator or blank)
Target Firmware	sonoff_zbdonglee_zigbee_router_8.0.2.0_115200_sw_flow.gbl
Flash Tool	universal-silabs-flasher (Python, pip-installable)
Bootloader Reset	rts_dtr (software reset via serial DTR/RTS lines)

2.3 V1 vs V2 Distinction

Feature	ZBDongle-P (V1)	ZBDongle-E (V2) — this dongle
Wireless Chip	Texas Instruments CC2652P	Silicon Labs EFR32MG21
USB-Serial Chip	CP2102N	CH9102
Firmware Format	.hex	.gbl
Flash Tool	cc2538-bsl / Z-Stack	universal-silabs-flasher
Driver	CP210x (built into most OS)	CH343SER (manual install on Windows)

Note: The coordinator dongle already running on the Pi is also a ZBDongle-E (EmberZNet, EZSP v13, firmware 7.4.4). This spare unit will be converted from coordinator/blank to **router** role.

3. Router Firmware Details

File	sonoff_zbdonglee_zigbee_router_8.0.2.0_115200_sw_flow.gbl
Repository	Nerivec/silabs-firmware-builder (community builds for Silicon Labs chips)
Firmware Version	8.0.2.0 (EmberZNet SDK)
Baud Rate	115200
Flow Control	Software (XON/XOFF)
Role	Zigbee Router — relays packets, extends mesh, always-on
Downloaded To	C:\Users\Sam\Downloads\ on Windows laptop

Note: The darkxst/silabs-firmware-builder repo does NOT have router firmware for ZBDongle-E — only NCP coordinator and OpenThread RCP images. Router firmware specifically comes from the **Nerivec** fork.

Why Router Firmware?

A Zigbee router is an always-powered device that relays messages between the coordinator and end devices. Battery-powered sensors (motion, presence, door/window, emergency button) are "sleepy end devices" — they don't relay for others and can only talk to their direct parent. By placing a router midway, distant sensors get a closer parent to talk to, reducing dropped messages and improving response time.

4. Chronological Steps Taken

Phase 1: Windows Laptop — Driver Installation

Step 1 — Downloaded CH343SER driver

Downloaded the correct WCH CH343SER driver package from the official WCH website. This driver covers the CH342/CH343/CH9102 chip family. Driver location: C:\Users\Sam\Downloads\CH343SER\ . INF file: CH343SER.INF. Installer shows: "USB-ENHANCED-SERIAL-A CH342".

Step 2 — Wrong driver attempt (CH341SER)

Initially opened the CH341SER installer by mistake. CH341SER covers the CH340 chip family, NOT the CH9102 in the ZBDongle-E V2. Identified and corrected — the correct driver is CH343SER.INF, not CH341SER.INF.

Step 3 — Pre-installed CH343SER driver

With the dongle unplugged, ran the CH343SER installer and clicked **INSTALL** (not UNINSTALL). Got confirmation: "The drive is successfully Pre-installed in advance!" — driver staged into Windows driver store.

Step 4 — Plugged in dongle — appeared under "Other devices"

Plugged the spare ZBDongle-E into the laptop. Windows showed "Setting up device" then "Device is ready", but the dongle appeared under **Other devices** in Device Manager, not under Ports (COM & LPT). Windows did not automatically match the CH343SER driver to Sonoff's specific USB VID/PID.

Step 5 — Automatic driver search failed

Right-clicked the device, chose Update driver, then "Browse my computer for drivers" — failed with "Windows could not find drivers for your device."

Step 6 — Manual driver assignment succeeded

Used the manual path: Update driver → "Let me pick from a list of available drivers" → device type: Ports (COM & LPT) → manufacturer: wch.cn → model: USB-Enhanced-SERIAL CH9102. Driver installed successfully.

Step 7 — Code 10 error on first USB port

Device appeared as COM3 under Ports but showed error: "This device cannot start. (Code 10)".

Step 8 — Moved to different USB port — Code 10 cleared

Unplugged the dongle and plugged into a different USB port. Code 10 error cleared. Device Manager showed USB-Enhanced-SERIAL CH9102 (COM3) under Ports — no warning icon, properties confirmed "This device is working properly".

Phase 2: Windows Laptop — Flash Attempt

Step 9 — Installed universal-silabs-flasher

```
pip install universal-silabs-flasher
```

Step 10 — Flash command executed

```
universal-silabs-flasher --device COM3 --bootloader-reset rts_dtr flash --firmware  
sonoff_zbdonglee_zigbee_router_8.0.2.0_115200_sw_flow.gbl
```

Step 11 — FileNotFoundError on COM3

Despite Device Manager showing a clean COM3 with no warnings, the flash tool failed:

```
FileNotFoundError: [Errno 2] could not open port '\\.\COM3': FileNotFoundError(2, 'The  
system cannot find the file specified.', None, 2)
```

Python's pyserial library also cannot open the port. The COM port registration is fundamentally broken at the Windows OS level despite appearing correct in Device Manager.

Phase 3: Raspberry Pi — Flash Attempt

Step 12 — Assessed Pi USB port situation

The Raspberry Pi has 4 USB ports: 2x USB 2.0 (both occupied) and 2x USB 3.0 (physically too tight for the dongle body).

Port	Type	Status
Port 1	USB 2.0	Occupied — Coordinator dongle (with extension cable)
Port 2	USB 2.0	Occupied — SSD SATA cable (boot drive)
Port 3	USB 3.0	Physically too tight for dongle body
Port 4	USB 3.0	Physically too tight for dongle body

Step 13 — SSD cannot be temporarily unplugged

The Pi boots from the SSD connected via USB 2.0 SATA adapter. Unplugging it would crash the running system. This rules out temporarily freeing a USB 2.0 port.

Step 14 — USB 3.0 physical constraint confirmed

USB 3.0 ports are electrically backward-compatible with the dongle, but the physical clearance between the two stacked USB 3.0 ports on the Pi is too tight for the ZBDongle-E's aluminium housing to fit without a USB extension cable.

5. Current Status & Blockers

5.1 Status Overview

Step	Status
Router firmware identified & downloaded	Done
Correct driver identified (CH343SER for CH9102)	Done
Driver installed on Windows laptop	Done
Device visible in Device Manager (COM3, no warnings)	Done
COM port actually functional on Windows	FAILED — <code>FileNotFoundException</code>
Pi direct flash attempt	BLOCKED — no physical USB space
Dongle flashed with router firmware	PENDING

5.2 Blocker A: Windows COM Port Non-Functional

Aspect	Status
Driver installed	Yes — CH343SER (CH9102)
Device Manager	Clean — COM3, no warning icon
Device Properties	"This device is working properly"
Actual COM port access	BROKEN — <code>FileNotFoundException</code> on <code>\\.\COM3</code>

The COM port appears correct in every visible way but cannot be opened by any software (pyserial, universal-silabs-flasher). This is a Windows-level issue where the port registration is superficially correct but non-functional at the OS level.

5.3 Blocker B: Pi USB Port Physical Space

Port	Status
USB 2.0 #1	Occupied — Coordinator dongle (with extension cable)
USB 2.0 #2	Occupied — SSD SATA cable (boot drive, cannot unplug)
USB 3.0 #1	Physically too tight for dongle's aluminium body
USB 3.0 #2	Physically too tight for dongle's aluminium body

No free port with adequate physical clearance to plug in the spare dongle directly.

6. Proposed Solutions (Next Steps)

Solution A: USB Extension Cable for Pi (Recommended)

Get a short USB-A male-to-female extension cable (even 10-15 cm is enough). Plug it into one of the Pi's USB 3.0 ports, then plug the spare dongle into the extension cable's female end — bypassing the physical clearance problem entirely.

Flash command on Pi:

```
# Find the new device ls /dev/ttyUSB* # Flash (assuming it appears as /dev/ttyUSB1)
universal-silabs-flasher --device /dev/ttyUSB1 \ --bootloader-reset rts_dtr flash \
--firmware /tmp/router_firmware.gbl
```

Why recommended: This is the simplest, most reliable approach. The Pi's Linux environment handles CH9102 natively (no driver issues like Windows), and universal-silabs-flasher works reliably on Linux. A USB extension cable costs under Rs. 100.

Solution B: Chrome Web Serial Flasher (Windows Alternative)

Use a browser-based flasher that accesses the COM port through Chrome's Web Serial API instead of Python's pyserial. The Web Serial API uses a different OS-level path to reach serial ports, which may bypass whatever is broken in Windows' COM port registration.

Options:

- **Nabu Casa SL Web Flasher** — supports custom .gbl upload for Silicon Labs devices
- **Darkxst Web Flasher** — community alternative at darkxst.github.io/silabs-firmware-builder/

Why alternative: Untested whether Chrome's Web Serial can reach the port when pyserial cannot, but it uses a fundamentally different code path and is worth trying before buying a cable.

7. Post-Flash Deployment Plan

1. **Power it** — with any standard USB charger (phone charger, power bank). The dongle only needs 5V/100mA. No host computer needed.
2. **Place it** — midway between the coordinator and the distant sensors. Ideally in a central hallway or room that bridges the coverage gap.
3. **Join it to the Zigbee network** — in Zigbee2MQTT, enable "Permit Join" and the router will automatically join the coordinator's network.
4. **Re-pair distant sensors** — remove and re-pair battery sensors while they are in their final position, so they discover and select the router as their parent node.
5. **Verify mesh** — in Zigbee2MQTT's network map, confirm sensors route through the new router node.

8. Flash Command Reference

On Raspberry Pi (Linux)

```
# Install flasher (if not already installed) pip install universal-silabs-flasher # Flash
router firmware universal-silabs-flasher \ --device /dev/ttyUSB1 \ --bootloader-reset rts_dtr
\ flash \ --firmware /tmp/sonoff_zbdonglee_zigbee_router_8.0.2.0_115200_sw_flow.gbl
```

On Windows (if COM port issue resolved)

```
universal-silabs-flasher --device COM3 --bootloader-reset rts_dtr flash --firmware
sonoff_zbdonglee_zigbee_router_8.0.2.0_115200_sw_flow.gbl
```

Valid Bootloader Reset Methods for ZBDongle-E

yellow, ihost, slzb07, rts_dtr, baudrate — **NOT** sonoff (that's for ZBDongle-P V1 only)

9. Summary

Item	Status
Dongle identified & verified	Done — ZBDongle-E V2, S/N: SL406010FC
Router firmware downloaded	Done — Nerivec v8.0.2.0 .gbl
Correct driver identified	Done — CH343SER for CH9102
Driver installed on Windows	Done — manual assignment to COM3
Windows flash attempt	Failed — COM3 not accessible at OS level
Pi flash attempt	Blocked — no physical USB port space
Current blocker	Need USB extension cable for Pi USB 3.0 port
Recommended next action	Get a short USB-A extension cable (Rs. 50-100)

The most reliable path forward is a USB extension cable to connect the spare dongle to a Pi USB 3.0 port. The Pi's Linux kernel has native CH9102 support and universal-silabs-flasher works without driver hassles on Linux. The Chrome Web Serial flasher is a zero-cost alternative to try on Windows in the meantime.